

LAPORAN PRAKTIKUM EXTRACTION OF FEATURE MAPS ON MNIST DATASET

Dosen Pengampu:

Bayu Adi Nugroho, Ph.D



**UIN SUNAN AMPEL
S U R A B A Y A**

Disusun Oleh:

Afrizal Dwi Ahmad – 09010624001

**PROGRAM STUDI SISTEM INFORMASI
FAKULTAS SAINS DAN TEKNOLOGI
UNIVERSITAS ISLAM NEGERI SUNAN AMPEL SURABAYA
2026**

1. Spesifikasi Perangkat dan Lingkungan Eksekusi

Berbeda dengan notebook aslinya yang dijalankan pada environment cloud Kaggle, pada praktikum ini kode dijalankan secara lokal menggunakan Jupyter Notebook pada laptop pribadi melalui Anaconda (environment bernama envGPU). Berikut adalah spesifikasi perangkat yang digunakan:

Komponen	Spesifikasi
Sistem Operasi	Kubuntu 26.04.1 LTS (Resolute Raccoon)
Host / Model Laptop	Lenovo LOQ 15IAX9 (83GS)
Processor (CPU)	Intel Core i5-12450HX
Graphics Card (GPU)	NVIDIA GeForce RTX 3050 6GB (Discrete GPU)
Memori (RAM)	32 GB

Perbedaan environment inilah yang menjadi penyebab utama munculnya berbagai error saat kode Kaggle direplikasi secara langsung, karena versi library Keras dan TensorFlow yang terpasang pada environment lokal jauh lebih baru (Keras 3.x) dibandingkan versi yang digunakan pada notebook Kaggle aslinya.

2. Sumber Kode Referensi

Kode program yang direplikasi pada praktikum ini bersumber dari notebook publik pada Kaggle dengan judul “Extraction of Feature Maps on MNIST Dataset”, yang dapat diakses melalui tautan berikut:

<https://www.kaggle.com/code/sanidhyajadaun/extraction-of-feature-maps-on-mnist-dataset/notebook>

Notebook tersebut membangun sebuah model CNN sederhana menggunakan Keras Sequential API untuk mengklasifikasikan angka pada dataset MNIST, kemudian mengekstrak dan memvisualisasikan feature maps dari beberapa layer konvolusi pada model yang telah dilatih.

3. Proses Replikasi dan Modifikasi Kode

Kode pada notebook Kaggle ditulis dan diuji pada environment Kaggle dengan versi Keras/TensorFlow yang relatif lebih lama, sehingga beberapa baris kode menggunakan API yang sudah deprecated (tidak lagi didukung) pada versi Keras 3.x yang terpasang

di environment lokal (envGPU). Oleh karena itu, beberapa bagian kode perlu dimodifikasi agar dapat berjalan dengan baik. Berikut penjelasan tiap perubahan yang dilakukan.

3.1 Perbaikan Parameter Optimizer Adam

Kode asli menggunakan parameter lr (singkatan dari learning rate) dan decay pada optimizer Adam, seperti berikut:

```
model.compile(optimizer=Adam(lr=.001, decay=1e-5),  
              loss='categorical_crossentropy',  
              metrics=['accuracy'])
```

Ketika dijalankan pada Keras 3.x, baris ini menghasilkan error `ValueError: Argument(s) not recognized: {'lr': 0.001}`. Hal ini terjadi karena pada Keras versi terbaru, parameter lr telah diganti namanya menjadi `learning_rate`, sedangkan parameter decay sudah sepenuhnya dihapus dari constructor optimizer dan tidak lagi didukung. Kode kemudian diperbaiki menjadi:

```
model.compile(optimizer=Adam(learning_rate=.001),  
              loss='categorical_crossentropy',  
              metrics=['accuracy'])
```

3.2 Penambahan Input Layer secara Eksplisit

Pada kode asli, shape input model didefinisikan secara implisit melalui parameter `input_shape` pada layer Dropout pertama:

```
model.add(Dropout(.25, input_shape=(width, height, 1), seed=1000))
```

Pendekatan ini merupakan cara lama (legacy) yang didukung pada versi Keras sebelumnya, namun pada Keras 3.x perilakunya menjadi tidak konsisten: model tidak benar-benar membentuk computational graph-nya (graph belum “dibangun”/built) hanya dengan cara ini, sehingga atribut `model.input` tidak dapat diakses meskipun model sudah pernah dilatih (fit) atau digunakan untuk prediksi (predict). Sebagai solusi, parameter `input_shape` dipindahkan menjadi layer Input eksplisit yang ditambahkan sebagai layer pertama pada model:

```
from keras import Input  
  
model = Sequential()  
model.add(Input(shape=(width, height, 1)))  
model.add(Dropout(.25, seed=1000))  
# ... layer-layer berikutnya tetap sama
```

Dengan cara ini, Keras langsung membentuk graph input/output model sejak model didefinisikan, tanpa harus bergantung pada proses training maupun prediksi terlebih dahulu.

3.3 Konsistensi Sumber Import (keras vs tensorflow.keras)

Seluruh import pada notebook (Sequential, Model, Dense, Conv2D, Adam, dan lain-lain) berasal dari package keras standalone (Keras 3, multi-backend), bukan dari tensorflow.keras. Pada salah satu percobaan perbaikan, layer Input sempat diimport dari sumber yang berbeda:

```
from tensorflow.keras import Input # tidak konsisten dengan import lain
```

Karena package keras dan tensorflow.keras merupakan dua instance module yang secara teknis terpisah pada environment ini, mencampur keduanya menyebabkan graph model tidak terhubung dengan benar walaupun secara tampilan kode terlihat benar. Perbaikannya adalah memastikan seluruh import, termasuk Input, berasal dari sumber yang sama:

```
from keras import Input # konsisten dengan import Sequential, Model, dsb.
```

3.4 Penggunaan model.inputs (bukan model.input)

Setelah Input layer eksplisit ditambahkan dan seluruh import telah konsisten, kode berikut pada bagian ekstraksi feature maps masih menghasilkan error AttributeError: The layer sequential has never been called and thus has no defined input:

```
feature_map_model = Model(inputs=model.input, outputs=model.layers[1].output)
```

Setelah ditelusuri lebih lanjut (termasuk dengan mereproduksi ulang kasus ini pada environment terpisah untuk memastikan penyebabnya), ditemukan bahwa hal ini merupakan perilaku khusus (quirk) pada Keras 3 ketika layer Input ditambahkan melalui model.add(Input(...)) pada model Sequential: atribut tunggal model.input gagal mengambil riwayat node meskipun model.built bernilai True dan atribut jamak model.inputs berfungsi dengan normal. Solusinya adalah mengganti model.input menjadi model.inputs (bentuk jamak) pada seluruh bagian kode yang membentuk model turunan (feature_map_model dan deep_layer):

```
feature_map_model = Model(inputs=model.inputs, outputs=model.layers[1].output)
...
deep_layer = Model(inputs=model.inputs, outputs=model.layers[9].output)
```

3.5 Perbaikan Duplikasi Layer dan Indeks Layer

Selama proses penyesuaian arsitektur model (penambahan Input layer), secara tidak sengaja beberapa baris model.add(...) pada blok Conv2D dengan 32 filter menjadi terduplikasi, sehingga arsitektur model memiliki lebih banyak layer daripada arsitektur aslinya. Akibatnya, indeks layer yang sebelumnya merujuk pada layer Activation dengan 64 feature map (digunakan pada bagian visualisasi deep layer, model.layers[9]) bergeser dan menunjuk ke layer lain yang hanya memiliki 32 feature

map, sehingga muncul error `IndexError: index 32 is out of bounds for axis 3 with size 32` saat proses plotting feature maps dengan konfigurasi grid 8x8 (64 feature map).

Penyebab ini dikonfirmasi dengan membangun ulang arsitektur model persis seperti pada kode yang bermasalah, lalu mencetak nama dan output shape dari setiap layer secara berurutan menggunakan perulangan `for` pada `model.layers`. Solusi yang diterapkan adalah menghapus baris `model.add(...)` yang terduplikasi pada blok Conv2D 32 filter, sehingga arsitektur model kembali sesuai dengan arsitektur pada notebook Kaggle asli (satu blok Conv2D 16 filter, satu blok Conv2D 32 filter, dan dua blok Conv2D 64 filter), dan indeks `model.layers[9]` kembali merujuk pada layer Activation dengan 64 feature map sebagaimana dimaksudkan oleh kode aslinya.

4. Ringkasan Error dan Solusi

Tabel berikut merangkum seluruh error yang ditemukan selama proses replikasi kode dari Kaggle ke environment lokal, beserta penyebab dan solusinya.

No	Error / Masalah	Penyebab dan Solusi
1	<code>ValueError: Argument(s) not recognized: {'lr': 0.001}</code>	Parameter <code>lr</code> pada optimizer Adam sudah deprecated dan diganti menjadi <code>learning_rate</code> ; parameter <code>decay</code> sudah tidak didukung lagi. Solusi: gunakan <code>Adam(learning_rate=.001)</code> dan hapus parameter <code>decay</code> .
2	<code>AttributeError: The layer sequential has never been called and thus has no defined input</code>	Shape input model didefinisikan secara implisit lewat parameter <code>input_shape</code> pada layer Dropout, sehingga graph model belum terbentuk. Solusi: tambahkan layer <code>Input(shape=...)</code> secara eksplisit sebagai layer pertama pada Sequential.
3	<code>AttributeError</code> yang sama masih muncul setelah Input layer ditambahkan	Layer Input diimpor dari <code>tensorflow.keras</code> , tidak konsisten dengan import Sequential/Model yang berasal dari package <code>keras standalone</code> . Solusi: impor <code>Input</code> dari <code>keras</code> agar konsisten dengan import lainnya.
4	<code>AttributeError</code> yang sama masih muncul walaupun import sudah konsisten	Keterbatasan (quirk) pada Keras 3: atribut tunggal <code>model.input</code> tidak berfungsi untuk Sequential yang Input layer-nya ditambahkan lewat <code>model.add()</code> . Solusi: gunakan <code>model.inputs</code> (bentuk jamak) sebagai pengganti <code>model.input</code> .
5	<code>IndexError: index 32 is out of bounds for axis 3 with size 32</code>	Terjadi duplikasi baris <code>model.add(Conv2D(filters=32, ...))</code> saat penyesuaian arsitektur, sehingga indeks layer bergeser dan <code>model.layers[9]</code> tidak lagi menunjuk pada layer dengan 64 feature map.

No	Error / Masalah	Penyebab dan Solusi
		Solusi: hapus baris Conv2D 32 filter yang terduplikasi agar arsitektur dan indeks layer kembali sesuai dengan kode asli.

5. Hasil Eksekusi

Setelah seluruh perbaikan di atas diterapkan, notebook dapat dijalankan secara berurutan (dari cell paling atas hingga cell paling bawah) tanpa error pada environment lokal (Kubuntu, Anaconda environment envGPU). Model CNN berhasil dilatih selama 3 epoch menggunakan data MNIST, dan feature maps berhasil diekstrak serta divisualisasikan pada dua tahap, yaitu:

- Feature maps dari layer konvolusi pertama (`model.layers[1]`), yang menghasilkan 16 feature map dan ditampilkan dalam grid 4x4.
- Feature maps dari layer konvolusi yang lebih dalam (`model.layers[9]`), yang menghasilkan 64 feature map dan ditampilkan dalam grid 8x8.

Hasil visualisasi menunjukkan bahwa feature maps pada layer awal cenderung menampilkan fitur berlevel rendah seperti garis tepi dan bentuk kasar dari angka pada citra input, sedangkan feature maps pada layer yang lebih dalam menampilkan pola yang lebih abstrak dan lebih sulit diinterpretasikan secara visual, sesuai dengan karakteristik hierarki representasi fitur pada CNN yang dijelaskan pada dasar teori.

6. Kesimpulan

Berdasarkan praktikum yang telah dilakukan, dapat disimpulkan bahwa proses replikasi kode dari platform cloud seperti Kaggle ke environment lokal tidak selalu berjalan mulus, meskipun kode yang direplikasi sudah terbukti berjalan dengan baik di sumber aslinya. Perbedaan versi library, khususnya Keras dan TensorFlow, dapat menyebabkan berbagai error akibat perubahan API antar versi (seperti perubahan nama parameter `lr` menjadi `learning_rate`, penghapusan parameter `decay`, serta perubahan perilaku pembentukan graph model pada Sequential API).

Selain itu, praktikum ini juga menunjukkan pentingnya konsistensi dalam penggunaan import library (dalam hal ini antara package `keras` standalone dan `tensorflow.keras`), serta ketelitian dalam menjaga urutan dan jumlah layer pada arsitektur model, karena kesalahan kecil seperti duplikasi satu blok layer dapat menyebabkan pergeseran indeks layer yang berdampak pada bagian kode lain yang mengacu pada indeks tersebut.

Dengan menyelesaikan seluruh error yang ditemukan secara bertahap dan sistematis, kode pada notebook Kaggle berhasil direplikasi dan dijalankan dengan baik pada perangkat laptop pribadi (Lenovo LOQ 15IAX9) dengan sistem operasi Kubuntu, sehingga tujuan praktikum untuk memahami proses ekstraksi dan visualisasi feature maps pada CNN dapat tercapai.